VIGNAN's INSTITUTE OF INFORMATION TECHNOLOGY
(AUTONOMOUS)
(Approved by AICTE-New Delhi & Affiliated to JNTUGV, Vizianagaram)
Beside VSEZ, Duvvada, Vadlapudi Post, Gajuwaka, Visakhapatnam - 530 049.

CLOUD COMPUTING LAB

(VR23)

[1046233110**]**

MANUAL & RECORD

B. TECH III YEAR – II SEM

[A.Y 2025-2026]

DEPARTMENT OF INFORMATION TECHNOLOGY

VIGNAN'S INSTITUTE OF INFORMATION TECHNLOGY(A)

DUVVADA, VISAKHAPATNAM - 530049

Vision of the Department:

To be a centre of excellence in high quality education and research producing globally competent IT professionals with ethical/human values to meet the needs of Information Technology sector and related services.

Mission of the Department:

M1: To impart high quality education through innovative teaching - learning practices resulting in strong software & hardware knowledge and skills to enable students to meet the challenges of IT profession.

M2: To facilitate faculty and students to carry out research work by providing necessary latest facilities and a conducive environment.

M3: To mould the students into effective professionals with necessary communication skills, team spirit, leadership qualities, managerial skills, integrity, social & environmental responsibility and lifelong learning ability with professional ethics and human values.

Program Educational Objectives (PEOs)

PEO1: To work in core IT companies/allied industries, educational institutions, research organizations and/or be entrepreneurs.

PEO2: To pursue higher education/ research in the field of Information Technology.

PEO3: To demonstrate communication skills, team spirit, leadership qualities, managerial skills, integrity, social & environmental responsibility and lifelong learning ability, professional ethics and human values in profession/career.

PSOs: Program Specific outcomes

PSO1: Analyze and design the solutions for data storage & computing systems.

PSO2: Implement the solutions for network and communication problems of Information Technology.

PROGRAM OUTCOMES

PO1 Engineering knowledge:

Apply the knowledge of mathematics science engineering fundamentals and mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems engineering problems.

PO2 Problem analysis:

Identify, formulate, review research Literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, and natural sciences, and engineering sciences.

PO3 Design/development of solutions:

Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural societal, and environmental considerations.

PO4 Conduct investigations of complex problems:

Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage:

Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6 The engineer and society:

Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Environment and sustainability:

Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development and need for sustainable development.

PO8 Ethics:

Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 Individual and team work:

Function effectively as an individual and as a member or leader in diverse teams and individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10 Communication:

Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project management and finance:

Demonstrate knowledge and understanding of the engineering and knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life-long learning:

Recognize the need for, and have the preparation and ability to engage in independent and life- ong learning in the broadest context of technological change.

III Year II Semester	CLOUD COMPUTING LAB	L	T	P	C
1046233110		0	0	3	1.5

Course Overview

Course Objectives: This course provides an insight into cloud computing. Topics covered include distributed system models, different cloud service models, service-oriented architectures, cloud programming and software environments, and resource management.

Course Outcomes: After the successful completion of this course, students will be able to:

Understand various service types, delivery models, and technologies of a cloud computing environment.

Understand the ways in which the cloud can be programmed and deployed.

Understand cloud service providers like CloudSim.

Examine various programming paradigms suitable to solve real-world and scientific problems using cloud services.

List of Experiments:

1. Install Virtual box/VMware Workstation with different flavors of Linux or windows OS on top of windows 7 o r10.
2. Install a C compiler in the virtual machine created using virtual box and execute Simple Programs
3. Create an AmazonEC2 instance and setup a web-server on the instance and associate an IP address with the instance.
4. Install GoogleAppEngine.Createahelloworldappandothersimplewebapplications using python/java.
5. Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

6. Find a procedure to transfer the files from one virtual machine to another virtual machine.
7. Write a Bash script in linux that performs the following tasks:
    a. Prints the message "Hello,World!" on the terminal.
    b. Lists all the files and directories in the current working directory
8. Perform IAM Operations Managing Users, Groups and Roles in AWSCloud.
9. Create a database instance in the cloud using Amazon RDS.
10. Managing persistent storage using amazon EBS

TEXTBOOK:

Essentials of cloud Computing: K.Chandrasekhran,CRCpress,2014

REFERENCEBOOKS:

Cloud Computing: Principles and Paradigms by Rajkumar Buyya,James Broberg and Andrzej M.Goscinski,Wiley,2011.

Distributed and Cloud Computing, Kai Hwang,Geoffery C.Fox,JackJ. Dongarra, Elsevier, 2012.

Cloud Computing Bible, Barrie Sosinsky,Wiley-India,2010

Experiment 1

Installation of VirtualBox and Ubuntu Linux

Aim

To install **VirtualBox** and install **Ubuntu Linux operating system** as a virtual machine.

Theory

Linux is an open-source operating system widely used in cloud computing environments.

Ubuntu is one of the most popular Linux distributions used in cloud platforms because of:

- Stability
- Security
- Open-source nature
- Large community support

Virtualization software such as Oracle VM VirtualBox or VMware Workstation allows multiple operating systems to run on one computer.

System Requirements

- Windows 10 / Windows 11
- 8GB RAM recommended
- 20GB free disk space
- Ubuntu ISO image

Procedure

Step 1: Download VirtualBox

Download from

https://www.virtualbox.org

Install using default settings.

Step 2: Download Ubuntu ISO

Download

https://ubuntu.com/download/desktop

Example version:

Ubuntu 22.04 LTS

Step 3: Create Virtual Machine

Open VirtualBox

Click **New**

Enter:

Name: Ubuntu_VM Type: Linux Version: Ubuntu (64-bit)

Step 4: Allocate Memory

Recommended

RAM = 4096 MB

Step 5: Create Virtual Hard Disk

Select

VDI

Dynamically allocated Size = 25 GB

Step 6: Attach Ubuntu ISO

Settings → Storage → Empty Disk

Select Ubuntu ISO file.

Step 7: Start Virtual Machine

Click **Start**

Ubuntu installation screen appears.

Choose:

Install Ubuntu

Follow instructions.

Expected Output

Ubuntu Linux desktop successfully installed.

Result

Successfully installed **Ubuntu Linux on VirtualBox**.

Experiment 2

Installing C Compiler in Virtual Machine and Executing Simple Programs

Aim

To install a **C compiler (GCC)** in a **virtual machine created using VirtualBox** and execute simple C programs.

Theory

A **compiler** is a system software that translates a program written in a **high-level programming language** into **machine code** so that the computer can execute it.

The **C compiler** most commonly used in Linux systems is the **GNU Compiler Collection (GCC)**. GCC compiles C source code into executable binary files.

A **Virtual Machine (VM)** is a software-based simulation of a computer system that allows multiple operating systems to run on a single physical machine. Tools such as **VirtualBox** or **VMware** create virtual environments where different operating systems like Linux can be installed and used without affecting the host system.

In Linux distributions such as Ubuntu, the C compiler can be installed using the package **build-essential**, which includes:

- GCC (C compiler)
- G++ (C++ compiler)
- make utility
- essential development libraries

Once installed, C programs can be written using a text editor, compiled using the **gcc command**, and executed through the terminal.

Procedure

Step 1: Start the Virtual Machine

1. Open **Oracle VirtualBox**.
2. Select the **Linux virtual machine** (e.g., Ubuntu).
3. Click **Start** to boot the VM.

Step 2: Open the Terminal

Open the terminal using:

Ctrl + Alt + T

Step 3: Update Package Repository

Update the system packages using the command:

sudo apt update

Enter the password when prompted.

Step 4: Install C Compiler

Install GCC compiler using:

sudo apt install build-essential

Press **Y** and press **Enter** to continue installation.

Step 5: Verify Compiler Installation

Check whether the compiler is installed successfully:

gcc --version

If installed correctly, the GCC version will be displayed.

Step 6: Create a C Program

Create a new file using nano editor:

```
nano hello.c
```

Write the following program:

```
#include <stdio.h>
int main()
{
   printf("Hello World\n");
   return 0;
}
```

Save the file and exit the editor.

Step 7: Compile the Program

Compile the program using the GCC compiler:

```
gcc hello.c -o hello
```

Step 8: Execute the Program

Run the compiled program using:

```
./hello
```

Results

The **GCC C compiler** was successfully installed in the **virtual machine**. The sample C program was compiled using the **gcc compiler** and executed successfully, producing the output:

Hello World

Thus, the installation of the C compiler and execution of simple C programs in the virtual environment was completed successfully.

Experiment 3

Creating an Amazon EC2 Instance and Setting up a Web Server

Aim

To create an **Amazon EC2 instance**, configure a **web server**, and associate a **public IP address** with the instance to host a webpage.

Theory

Amazon Elastic Compute Cloud (EC2) is a web service provided by Amazon Web Services that allows users to launch and manage virtual servers in the cloud. These servers are called **instances** and can run various operating systems such as Linux or Windows.

An EC2 instance provides scalable computing capacity, enabling developers and organizations to run applications without purchasing physical hardware. Each instance can be assigned a **public IP address** that allows users to access services hosted on the instance via the internet.

A **web server** is software that handles HTTP requests from clients and delivers web pages. Common web servers used on Linux systems include:

- Apache HTTP Server
- Nginx

In this experiment, a Linux-based EC2 instance is launched, a web server is installed, and a webpage is hosted on it. The instance is then accessed through its public IP address using a web browser.

Procedure

Step 1: Login to AWS Console

1. Open a web browser.
2. Go to the AWS Management Console.
3. Login to your account in Amazon Web Services.

Step 2: Open EC2 Service

1. From the AWS dashboard, select **Services**.
2. Choose **EC2 (Elastic Compute Cloud)**.

Step 3: Launch a New Instance

1. Click **Launch Instance**.
2. Enter the **instance name** (Example: *WebServerInstance*).
3. Select an operating system such as **Ubuntu Server** or **Amazon Linux**.

Step 4: Choose Instance Type

Select a free-tier eligible instance type:

- **t2.micro** (commonly used for practice)

Step 5: Create Key Pair

1. Click **Create new key pair**.
2. Enter a key name (example: *webkey*).
3. Download the **.pem file** and save it securely.

This key is used for **SSH login to the server**.

Step 6: Configure Network Settings

1. Allow the following inbound rules:

Type	Port
SSH	22
HTTP	80
HTTPS	443

This allows remote login and web access.

Step 7: Launch the Instance

1. Click **Launch Instance**.
2. Wait for the instance to start.
3. Note the **Public IPv4 Address** assigned to the instance.

Step 8: Connect to the Instance

Open terminal and connect using SSH:

ssh -i webkey.pem ubuntu@<Public-IP-Address>

Example:

ssh -i webkey.pem ubuntu@54.210.10.25

Step 9: Install Web Server

Update the system:

sudo apt update

Install **Apache Web Server**:

sudo apt install apache2

Start the server:

sudo systemctl start apache2

Enable it to run automatically:

sudo systemctl enable apache2

Step 10: Create a Web Page

Edit the default webpage:

sudo nano /var/www/html/index.html

Example content:

```
<html>
<head>
<title>My Cloud Server</title>
</head>
<body>
<h1>Welcome to My AWS EC2 Web Server</h1>
<p>Server is running successfully.</p>
</body>
</html>
```

Save the file.

Step 11: Access the Web Server

1. Copy the **Public IP Address** of the instance.
2. Open a web browser.
3. Enter:

http://Public-IP-Address

Example:

http://54.210.10.25

The webpage will be displayed.

Results

An **Amazon EC2 instance** was successfully created in the Amazon Web Services cloud platform. A **web server (Apache)** was installed and configured on the instance. The server was accessed successfully using the **public IP address**, and the hosted webpage was displayed in the browser.

Experiment 4

Installing Google App Engine and Creating a Hello World Web Application

Aim

To install **Google App Engine** and create a **Hello World web application** and other simple web applications using **Python/Java**.

Theory

Google provides a cloud computing platform called **Google App Engine (GAE)** which allows developers to build and host web applications on Google's infrastructure. It is a **Platform as a Service (PaaS)** that manages server infrastructure automatically.

Developers only need to upload their application code, and the platform handles:

- Server provisioning
- Load balancing
- Scaling
- Application deployment

Applications developed in **Python, Java, Go, and other languages** can be deployed on App Engine. It uses a configuration file called **app.yaml** (for Python applications) that defines the application runtime and request handling.

A simple **Hello World application** is commonly used to test whether the platform and development environment are installed correctly.

Using Google App Engine offers several advantages:

- Automatic scaling
- High availability
- Built-in security
- Easy deployment

Procedure

Step 1: Install Python / Java

Install Python or Java on the system.

To check Python installation:

python --version

To check Java installation:

java -version

Step 2: Install Google Cloud SDK

Download and install **Google Cloud SDK** from the official website of Google.

After installation, initialize the SDK:

gcloud init

Login with your Google account and select or create a cloud project.

Step 3: Create App Engine Application

Create an App Engine project using:

gcloud app create

Select the **region** for deployment.

Creating a Hello World Application (Python)

Step 4: Create Project Directory

```
mkdir helloworld
cd helloworld
```

Step 5: Create Python Application

Create a file named **main.py**

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello World from Google App Engine!"

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=True)
```

Step 6: Create Configuration File

Create **app.yaml**

```
runtime: python39

handlers:
- url: /.*
  script: auto
```

Step 7: Deploy Application

Deploy the application using:

```
gcloud app deploy
```

Step 8: Run the Application

Open the application in the browser using:

gcloud app browse

Output displayed in browser:

Hello World from Google App Engine!

Example of Another Simple Web Application

Create another Python application file **app2.py**

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to My Cloud Web Application"

@app.route('/about')
def about():
    return "This application is running on Google App Engine"

if __name__ == '__main__':
    app.run()
```

This program demonstrates multiple web routes.

Results

The Google App Engine environment was successfully installed using the **Google Cloud SDK**. A **Hello World web application** was developed and deployed using **Python**, and the application was successfully accessed through a web browser. Additional simple web applications were also created and executed successfully.

Experiment 5

Simulation of a Cloud Scenario using CloudSim and Implementation of a Scheduling Algorithm

Aim

To simulate a **cloud computing environment using CloudSim** and implement a **custom task scheduling algorithm that is not present in the default CloudSim framework**.

Theory

CloudSim is an open-source simulation toolkit developed for modeling and simulating **cloud computing infrastructures and services**. It allows researchers and developers to test resource provisioning algorithms, scheduling techniques, and cloud application performance without using real cloud environments.

CloudSim supports modeling of the following components:

- **Data Centers** – Infrastructure providers
- **Hosts** – Physical machines
- **Virtual Machines (VMs)** – Virtualized computing resources
- **Cloudlets** – Tasks or jobs submitted by users
- **Brokers** – Components that schedule tasks to virtual machines

Scheduling algorithms determine **how tasks are assigned to virtual machines** in a cloud environment. CloudSim already provides scheduling policies such as:

- First Come First Serve (FCFS)
- Round Robin
- Time Shared Scheduling

However, researchers can extend CloudSim to implement **custom scheduling algorithms** such as:

- Priority Scheduling

- Shortest Job First
- Min-Min
- Max-Min
- Genetic Algorithm based scheduling

In this experiment, a **custom scheduling algorithm (e.g., Priority-based scheduling)** is implemented and evaluated using CloudSim.

Procedure

Step 1: Install Required Software

Install the following tools:

- **Java Development Kit (JDK)**
- **CloudSim toolkit**
- **Eclipse IDE or any Java IDE**

Step 2: Setup CloudSim Project

1. Download CloudSim toolkit.
2. Extract the CloudSim package.
3. Open **Eclipse IDE**.
4. Create a **new Java project**.
5. Import all **CloudSim libraries (.jar files)** into the project.

Step 3: Create Data Center

Create a **data center** that contains hosts and processing elements.

Example code snippet:

```
Datacenter datacenter0 = createDatacenter("Datacenter_0");
```

The data center simulates the cloud infrastructure.

Step 4: Create Datacenter Broker

The **broker** acts as a mediator between users and cloud resources.

```
DatacenterBroker broker = new DatacenterBroker("Broker");
int brokerId = broker.getId();
```

Step 5: Create Virtual Machines

Virtual machines simulate computing resources.

Example:

```
Vm vm1 = new Vm(0, brokerId, 1000, 1, 512, 1000, 1000, "Xen", new
CloudletSchedulerTimeShared());
```

Multiple VMs can be created and added to a list.

Step 6: Create Cloudlets (Tasks)

Cloudlets represent user jobs submitted to the cloud.

Example:

```
Cloudlet cloudlet1 = new Cloudlet(
    id, length, pesNumber, fileSize, outputSize,
    utilizationModel, utilizationModel, utilizationModel);
```

Add cloudlets to a list.

Step 7: Implement Custom Scheduling Algorithm

Example: **Priority Scheduling Algorithm**

Instead of default scheduling, cloudlets are assigned based on **priority level**.

Example logic:

```
Collections.sort(cloudletList, new Comparator<Cloudlet>() {
    public int compare(Cloudlet c1, Cloudlet c2) {
        return Long.compare(c1.getCloudletLength(), c2.getCloudletLength());
    }
});
```

This sorts tasks based on execution length (Shortest Job First style).

The sorted tasks are then assigned to virtual machines.

Step 8: Start Simulation

Initialize and run the simulation.

```
CloudSim.startSimulation();
CloudSim.stopSimulation();
```

Step 9: Display Output Results

Print the execution details:

- Cloudlet ID
- VM ID
- Start Time
- Finish Time

Example:

```
printCloudletList(newList);
```

Results

The **cloud computing environment** was successfully simulated using **CloudSim**. A **custom scheduling algorithm** was implemented and executed in the simulation. The scheduling algorithm successfully assigned cloudlets to virtual machines and produced execution results showing the task allocation and completion times.

Experiment 6

Transferring Files from One Virtual Machine to Another Virtual Machine

Aim

To study and implement different methods for **transferring files from one virtual machine to another virtual machine**.

Theory

A **Virtual Machine (VM)** is a software-based computer system that runs an operating system and applications just like a physical computer. Virtual machines are commonly created using virtualization software such as **Oracle VM VirtualBox** or **VMware Workstation**.

Sometimes it is necessary to transfer files between virtual machines for purposes such as:

- Data sharing
- Software installation
- Application deployment
- System configuration

File transfer between virtual machines can be performed using several techniques depending on network configuration and operating system support. Common methods include:

1. **Using Shared Folders**
2. **Using Secure Copy Protocol (SCP)**
3. **Using File Transfer Protocol (FTP)**
4. **Using Network File Sharing (Samba/NFS)**

In most Linux-based virtual machines, **SCP or SSH** is widely used because it provides **secure encrypted file transfer over the network**.

Procedure

Method 1: File Transfer Using SCP (Secure Copy)

Step 1: Connect Both Virtual Machines to the Same Network

1. Open **Virtual Machine Settings**.
2. Set the **Network Adapter** to **Bridged Adapter** or **NAT Network**.
3. Ensure both VMs can communicate with each other.

Step 2: Find IP Address of the Destination VM

Open terminal in the destination VM and run:

ifconfig

or

ip addr

Note the **IP address** of the destination VM.

Example:

192.168.1.15

Step 3: Transfer File Using SCP

From the **source virtual machine**, run:

scp filename user@destination_IP:/home/user/

Example:

scp file.txt user@192.168.1.15:/home/user/

Enter the **password of the destination VM** when prompted.

Step 4: Verify File Transfer

Login to the destination VM and check:

ls

The transferred file will appear in the specified directory.

Method 2: File Transfer Using Shared Folder (VirtualBox)

Step 1: Open Virtual Machine Settings

1. Select the VM in **Oracle VM VirtualBox.
2. Go to **Settings → Shared Folders**.

Step 2: Add a Shared Folder

1. Click **Add Shared Folder**.
2. Select a folder from the host system.
3. Enable **Auto-mount** and **Make Permanent**.

Step 3: Access Shared Folder in VM

Inside the VM terminal, access the shared folder:

cd /media

or

cd /mnt

Files placed in this folder will be accessible to both virtual machines.

Method 3: File Transfer Using FTP

1. Install FTP server on destination VM:

sudo apt install vsftpd

2. Start the FTP service:

sudo systemctl start vsftpd

3. Transfer files using an FTP client.

Results

Files were successfully transferred from one **virtual machine** to another using **network-based file transfer techniques** such as **SCP and shared folders**. The experiment demonstrated secure and efficient data transfer between virtual environments created using virtualization platforms.

Experiment 7

Writing a Bash Script in Linux

Aim

To write and execute a **Bash script in Linux** that performs the following tasks:

1. Prints the message **"Hello, World!"** on the terminal.
2. Lists all the **files and directories in the current working directory**.

Theory

A **Bash script** is a text file containing a sequence of commands that are executed by the **Bash shell**. Bash stands for **Bourne Again Shell** and is the default command interpreter for most Linux systems.

Shell scripting helps automate repetitive tasks such as:

- File management
- System monitoring
- Process control
- Program execution

A Bash script typically begins with a **shebang line**:

#!/bin/bash

This line tells the operating system that the script should be executed using the **Bash interpreter**.

Common commands used in this experiment:

- echo → Prints text on the terminal
- ls → Lists files and directories in a directory

By combining these commands in a script file, multiple tasks can be performed automatically.

Procedure

Step 1: Open Terminal

Open the Linux terminal using:

Ctrl + Alt + T

Step 2: Create a Script File

Create a file named **hello.sh**

nano hello.sh

Step 3: Write the Bash Script

Type the following code in the file:

```
#!/bin/bash

# Print Hello World message
echo "Hello, World!"

# List files and directories
echo "Files and directories in current working directory:"
ls
```

Step 4: Save the File

Press:

CTRL + O

Press **Enter**, then press:

CTRL + X

Step 5: Provide Execute Permission

Make the script executable:

chmod +x hello.sh

Step 6: Run the Script

Execute the script using:

./hello.sh

Output

Example output on terminal:

Hello, World!
Files and directories in current working directory:
Documents
Downloads
file1.txt
script.sh
Pictures

Results

The Bash script was successfully created and executed. The script printed the message **"Hello, World!"** and displayed the **list of files and directories in the current working directory**.

Experiment 8

Perform IAM Operations – Managing Users, Groups, and Roles in AWS Cloud

Aim

To perform **Identity and Access Management (IAM) operations** in the AWS Cloud by creating and managing **users, groups, and roles**.

Theory

Identity and Access Management (IAM) is a service provided by Amazon Web Services that enables administrators to securely manage access to AWS resources.

Using **AWS Identity and Access Management (IAM)**, administrators can:

- Create users who can access AWS resources.
- Organize users into groups.
- Assign permissions using policies.
- Create roles for services and applications.

Key IAM Components

1. Users

- Individual identities created within AWS.
- Each user has unique login credentials.

2. Groups

- A collection of IAM users.
- Permissions can be assigned to the group instead of individual users.

3. Roles

- Temporary permissions assigned to AWS services or external users.
- Roles do not have permanent credentials.

IAM helps enforce the **principle of least privilege**, meaning users only get the permissions necessary to perform their tasks.

Procedure

Step 1: Login to AWS Console

1. Open a web browser.
2. Go to the AWS Management Console.
3. Login to your account in Amazon Web Services.

Creating an IAM User

Step 2: Open IAM Service

1. In the AWS dashboard, click **Services**.
2. Select **IAM (Identity and Access Management)**.

Step 3: Create a New User

1. Click **Users → Add Users**.
2. Enter a **User Name** (Example: *CloudUser1*).
3. Select **Access Type**:
   - AWS Management Console Access
   - Programmatic Access
4. Set a password.

Click **Next**.

Creating an IAM Group

Step 4: Create a Group

1. Click **Create Group**.

2. Enter **Group Name** (Example: *Developers*).
3. Attach policies such as:
    - **AmazonEC2FullAccess**
    - **AmazonS3FullAccess**

Click **Create Group**.

Step 5: Add User to Group

1. Select the created user.
2. Click **Add user to group**.
3. Choose the group **Developers**.

Click **Next → Create User**.

Creating an IAM Role

Step 6: Create Role

1. In IAM Dashboard, click **Roles**.
2. Click **Create Role**.
3. Select **AWS Service** as trusted entity.
4. Choose service (Example: **EC2**).

Click **Next**.

Step 7: Attach Permissions to Role

Attach a policy such as:

- **AmazonS3ReadOnlyAccess**

Click **Next**.

Step 8: Name the Role

Provide a role name:

Example:

EC2-S3-AccessRole

Click **Create Role**.

Verifying IAM Configuration

1. Login using the created **IAM user credentials**.
2. Verify access permissions.
3. Confirm that permissions are applied through **groups and roles**.

Results

IAM operations were successfully performed in the Amazon Web Services cloud environment. Users, groups, and roles were created using **AWS Identity and Access Management**, and appropriate access permissions were assigned successfully.

Experiment 9

Creating a Database Instance in Cloud using Amazon RDS

Aim

To create a **database instance in the cloud using Amazon RDS** and access it for database operations.

Theory

Amazon Relational Database Service (RDS) is a cloud-based database service provided by Amazon Web Services. It allows users to easily set up, operate, and scale relational databases in the cloud.

Using **Amazon Relational Database Service**, administrators can create and manage databases without worrying about hardware provisioning, database setup, patching, backups, and scaling.

Amazon RDS supports several database engines such as:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- Microsoft SQL Server
- Amazon Aurora

Key Features of Amazon RDS

- Automated backups
- Database scaling
- Security and access control
- High availability
- Monitoring and maintenance

RDS allows users to launch **database instances**, which are isolated database environments running in the cloud.

Procedure

Step 1: Login to AWS Console

1. Open a web browser.
2. Go to the AWS Management Console.
3. Login to your account in Amazon Web Services.

Step 2: Open RDS Service

1. From the AWS dashboard, click **Services**.
2. Select **RDS (Relational Database Service)**.

Step 3: Create Database

1. Click **Create Database**.
2. Select **Standard Create**.

Step 4: Choose Database Engine

Select a database engine such as:

- **MySQL**

Supported by **Amazon Relational Database Service**.

Click **Next**.

Step 5: Choose Instance Type

Select:

- **Free tier template**

Instance class:

db.t2.micro

This is suitable for learning and testing.

Step 6: Configure Database Settings

Enter the following details:

Parameter	Example
DB Instance Identifier	mydatabase
Master Username	admin
Master Password	password123

Step 7: Configure Storage

Choose:

- Storage Type: **General Purpose SSD**
- Allocated Storage: **20 GB**

Step 8: Configure Connectivity

1. Choose **Default VPC**.
2. Set **Public Access = Yes**.
3. Configure security group to allow **MySQL port 3306**.

Step 9: Launch Database

1. Click **Create Database**.
2. Wait until the database status becomes **Available**.

Step 10: Connect to Database

Note the **Endpoint URL** of the RDS instance.

Example:

mydatabase.xxxxxx.us-east-1.rds.amazonaws.com

Connect using MySQL client:

mysql -h endpoint -u admin -p

Enter the password when prompted.

Step 11: Verify Database

Run a simple SQL command:

SHOW DATABASES;

Results

A **database instance was successfully created in the cloud using Amazon Relational Database Service** provided by Amazon Web Services. The database was configured with required parameters and accessed successfully using the database endpoint.

Experiment 10

Managing Persistent Storage using Amazon EBS

Aim

To create and manage **persistent storage using Amazon EBS** and attach it to an EC2 instance in the AWS cloud environment.

Theory

Amazon Elastic Block Store (EBS) is a block-level storage service provided by Amazon Web Services. It is designed to provide persistent storage for virtual machines running on **Amazon EC2**.

EBS volumes behave like **virtual hard disks** that can be attached to EC2 instances. The data stored in EBS persists even if the instance is stopped or terminated.

Key features of **Amazon Elastic Block Store** include:

- Persistent block storage
- High availability and reliability
- Data backup through snapshots
- Flexible storage sizes
- Easy attachment and detachment from EC2 instances

Types of EBS Volumes

1. **General Purpose SSD (gp2/gp3)** – Balanced performance
2. **Provisioned IOPS SSD** – High performance databases
3. **Throughput Optimized HDD** – Big data workloads
4. **Cold HDD** – Low-cost storage for infrequent access

EBS volumes are commonly used for:

- Databases

- File systems
- Application data storage

Procedure

Step 1: Login to AWS Console

1. Open a web browser.
2. Go to the AWS Management Console.
3. Login to your account in Amazon Web Services.

Step 2: Launch EC2 Instance

1. Navigate to **EC2 Dashboard**.
2. Click **Launch Instance**.
3. Select an operating system such as **Amazon Linux or Ubuntu**.
4. Choose instance type **t2.micro**.
5. Launch the instance.

Step 3: Create an EBS Volume

1. In EC2 Dashboard, click **Elastic Block Store → Volumes**.
2. Click **Create Volume**.

Enter the following details:

Parameter	**Example**
Volume Type	General Purpose SSD
Size	10 GB
Availability Zone	Same as EC2 instance

3. Click **Create Volume**.

Step 4: Attach Volume to EC2 Instance

1. Select the created EBS volume.
2. Click **Actions → Attach Volume**.
3. Choose the EC2 instance.
4. Specify device name:

/dev/sdf

5. Click **Attach**.

Step 5: Connect to EC2 Instance

Use SSH to connect:

ssh -i key.pem ec2-user@Public-IP

Step 6: Check Attached Disk

Run the following command:

lsblk

The new disk will appear as **xvdf**.

Step 7: Create File System

Format the disk:

sudo mkfs -t ext4 /dev/xvdf

Step 8: Create Mount Directory
sudo mkdir /data

Step 9: Mount the Volume

sudo mount /dev/xvdf /data

Verify the mount:

df -h

Step 10: Store Data in EBS

Create a test file:

sudo touch /data/testfile.txt

Results

Persistent storage was successfully created using **Amazon Elastic Block Store** in the cloud environment provided by **Amazon Web Services**. The EBS volume was attached to an **Amazon EC2** instance, formatted, mounted, and used to store data successfully.